**Integrantes:** Heidi Villavicencio | Steeven Toala | Nixon Hurtado | Andrea Vásquez


## 1. TÍTULO

**Denominación del Entregable:** Desarrollo del Wizard Modular Asistido por Pasos para Elaboración de Planes e Informes (React + Vite, Tailwind CSS Keyframes #00345E)

## 2. INTRODUCCIÓN

Los formatos institucionales del Sistema de Gestión de Calidad (SGC) de la FISEI UTA (`UTA-SGC-A-2-1-P7-T1` para el Plan de Trabajo e `UTA-SGC-A-2-1-P7-T2` para el Informe de Cumplimiento) se componen de múltiples secciones secuenciales: Datos Informativos, Justificación/Objetivos, Matriz Cronograma de Actividades, Anexos/Evidencias y Circuito de Firmas. La presentación de estos extensos formularios en una sola vista monolítica satura al usuario y propicia la pérdida involuntaria de datos.

Para resolver esta problemática de interacción, se diseñó e implementó un componente asistido por pasos (**Wizard Modular**) en **React + Vite** y **Tailwind CSS**. Este asistente divide el proceso de llenado en pestañas lógicas conectadas, valida los requisitos mínimos de cada etapa antes de permitir la navegación, proporciona un indicador de avance porcentual en tiempo real y gestiona el estado transaccional del expediente con soporte para microanimaciones CSS Keyframes.

## 3. OBJETIVO

### 3.1. Objetivo General

Desarrollar el componente de interfaz `PlanEditorWizard.jsx` bajo la arquitectura de pestañas modulares en React y Vite, guiando al docente en el llenado guiado y secuencial de su planificación semestral conforme al formato oficial del SGC de la FISEI UTA.

### 3.2. Objetivos Específicos

- Diseñar la barra de navegación por pasos con estados visuales claros: *Completado*, *En Edición*, *Pendiente* y *Bloqueado*, aplicando los colores institucionales (`#00345E`).
- Implementar validaciones deterministas de salida en cada pestaña (Validation Gates que impiden avanzar si faltan campos obligatorios).
- Integrar el cálculo reactivo del porcentaje de completitud global del expediente.
- Conectar el Wizard con el cliente Axios institucional con auto-renovación de tokens y el backend Node.js + Express (v5).

## 4. ALCANCE

### 4.1. Alcance Incluido

Componente `PlanEditorWizard.jsx` ubicado en `frontend/src/pages/`, Pestaña 1 (Datos Informativos), Pestaña 2 (Justificación y Objetivos), Pestaña 3 (Contenedor de la Matriz de Actividades y Ponderaciones al 100.00 %), Pestaña 4 (Anexos y Evidencias) y Pestaña 5 (Vista previa y Cascada de Firmas).

### 4.2. Límites y Exclusiones

El editor enriquecido de texto con autoguardado se encapsula en EDT 4.2; la exportación oficial a PDF/Word se encapsula en EDT 4.3.

## 5. DEFINICIONES

- **Wizard Pattern:** Patrón de diseño de interfaz que descompone una tarea compleja en pasos secuenciales y validados.
- **Validation Gate:** Condición de negocio que restringe el paso a la siguiente etapa hasta satisfacer los requisitos de calidad.
- **EDT (*Estructura de Desglose del Trabajo*):** Descomposición funcional jerárquica de entregables del proyecto.

## 6. RESPONSABILIDADES

- **Andrea Vasquez (Frontend Developer / QA):** Programación del componente `PlanEditorWizard.jsx`, manejo de estado y validadores.
- **Heidy Villavicencio (UI-UX):** Ajuste estético en Tailwind CSS, paleta `#00345E`, animaciones Keyframes e iconos de estado.
- **Ing. Paulo Torres (Docente Tutor):** Supervisión técnica y validación del diseño UX/UI institucional.

## 7. NORMATIVA LEGAL Y TÉCNICA

- **Guía de Estilo Visual FISEI UTA:** Paleta institucional (`#00345E`), contraste tipográfico WCAG 2.1 AA y diseño responsive.
- **ISO 9241-11:** Ergonomía de la interacción persona-sistema (Usabilidad, Eficacia y Satisfacción).
- **Norma Técnica SGC UTA:** Formatos institucionales `UTA-SGC-A-2-1-P7-T1` y `UTA-SGC-A-2-1-P7-T2`.

## 8. MÉTODO, POLÍTICAS Y CONTENIDOS

Construcción de componentes funcionales modulares en React con Hooks nativos y Tailwind CSS. Se establecen políticas rigurosas de navegación:

1. El docente puede regresar libremente a pasos anteriores guardados.
2. Se bloquea el botón ”Siguiente”si el paso actual no satisface la totalidad de validaciones requeridas.
3. La pestaña de firmas permanece bloqueada hasta que la matriz de ponderación cierre con exactitud en el 100.00 %.

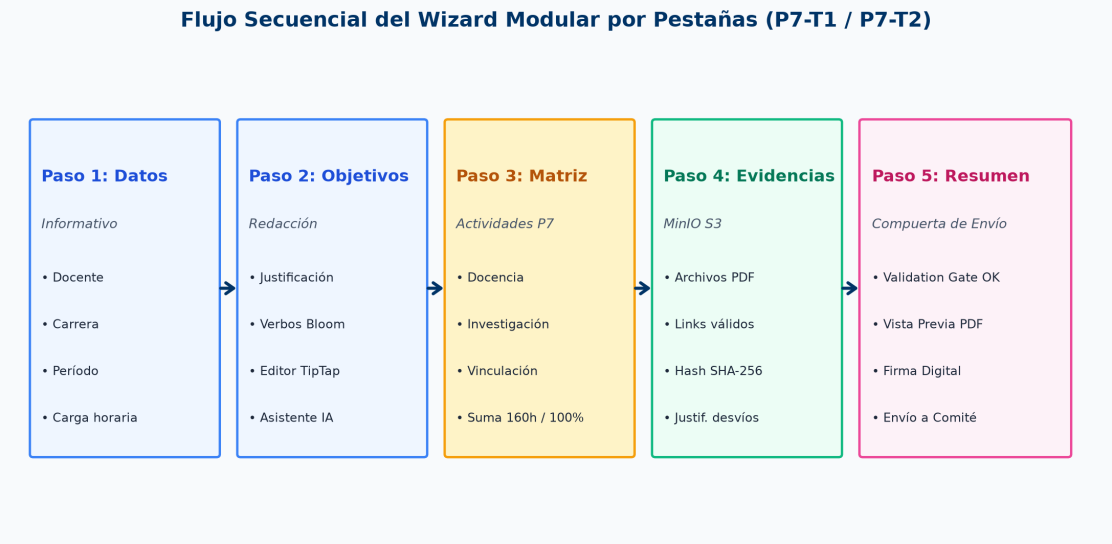



Figura 1: Flujo Secuencial del Wizard Modular por Pestañas (P7-T1 / P7-T2).

## 9. DESCRIPCIÓN DE ACTIVIDADES

### 9.1. Estructura de Pasos del Wizard (PlanEditorWizard.jsx)

```
// frontend/src/pages/PlanEditorWizard.jsx
export const WIZARD_STEPS = [
  { id: 1, key: 'datos', label: '1. Datos Informativos', icon: 'FileText
     ' },
  { id: 2, key: 'justificacion', label: '2. Justificacion y Objetivos',
     icon: 'Target' },
  { id: 3, key: 'matriz', label: '3. Matriz Cronograma (100%)', icon: '
     Table' },
  { id: 4, key: 'evidencias', label: '4. Anexos y Evidencias', icon: '
     Paperclip' },
```

```
  { id: 5, key: 'firmas', label: '5. Cascada de Firmas', icon: '
      CheckSquare' }
];
```

### 9.2. Flujo de Validación y Completitud Reactiva

El avance porcentual se calcula reactivamente en memoria:

$$\text{Progreso Global} = \frac{\sum_{i=1}^{5} \text{PuntajePaso}_i}{5} \times 100\,\% \tag{1}$$

Si el docente intenta acceder a la pestaña de Firmas sin alcanzar el 100 % de ponderación en la Matriz, la interfaz despliega una alerta modal bloqueante con la advertencia de Validation Gate.

### 9.3. Conclusiones y Recomendaciones Técnicas

- El Wizard modular elimina la fatiga cognitiva del docente frente a formularios largos, guiándolo paso a paso y reduciendo los rechazos por omisión de requisitos.
- La arquitectura por componentes facilita la reutilización inmediata tanto para planes de trabajo como para informes finales de cumplimiento.
- Se recomienda articular el Wizard con el motor de autoguardado periódico TipTap (EDT 4.2) para brindar resiliencia absoluta ante caídas de red.

## 10. FIRMAS DE RESPONSABILIDAD

| ACCIÓN | NOMBRE Y CARGO | FIRMA | FECHA |
|---|---|---|---|
| **Elaborado por:** | Heidi Villavicencio<br>Analista de Requisitos / UI-UX | | 23/09/2026 |
| **Revisado por:** | Steeven Toala<br>Líder de Proyecto / Arquitecto | | 23/09/2026 |
| **Aprobado por:** | Ing. Paulo Torres, Mg.<br>Docente Evaluador – Gestión Proyectos | [ PENDIENTE DE APROBACIÓN ] | **Pendiente** |

| | | |
|---|---|---|
| SISTEMA DE GESTIÓN DE LA CALIDAD | UNIVERSIDAD TÉCNICA DE AMBATO | |
| | PLAN DE TRABAJO:..... | |
| | Unidad académica/ administrativa: | |
| | Fecha de elaboración: | |

# UNIVERSIDAD TÉCNICA DE AMBATO

**UNIDAD ACADÉMICA / ADMINISTRATIVA: …**

**PLAN DE TRABAJO DE: ……**

**PERÍODO: …….**



Figura 2: Captura Real de la Interfaz del Asistente Wizard en Ejecución (Frontend React 19).

| | | |
|---|---|---|
| SISTEMA DE GESTIÓN DE LA CALIDAD | **UNIVERSIDAD TÉCNICA DE AMBATO** | |
| | **PLAN DE TRABAJO:.....** | |
| | Unidad académica/ administrativa: | |
| | Fecha de elaboración: | |

## ÍNDICE DE CONTENIDO

## ÍNDICE DE TABLAS



Figura 3: Captura de Interfaz: Pestaña de Justificación y Objetivos Académicos con Auto-validación.

## 11. CONTROL DE CAMBIOS

| Versión | Descripción del cambio | Responsable del cambio | Fecha de aprobación |
|---|---|---|---|
| 1.0 | Creación inicial del componente Wizard modular. | Andrea Vasquez | 16/09/2026 |
| 2.0 | Actualización integral al stack oficial: React + Vite, Tailwind CSS Keyframes #00345E e integración con Axios centralizado. | Andrea Vasquez / Heidy Villavicencio | 16/09/2026 |

## 12. ANEXOS

### Anexo 1: Referencias Bibliográficas


- Nielsen Norman Group. (2024). *Wizards: Definition and Design Recommendations*.
- React Community. (2026). *React Component Architecture and Custom Hooks*.
- Tailwind Labs. (2026). *Building Accessible Steppers with Tailwind CSS*.


### Anexo 2: Flujo Lógico de Validación por Pasos

Diagrama de estados de cada pestaña y transiciones permitidas según el cumplimiento de las Validation Gates del SGC FISEI UTA.